

Position-Aware Chunking for Industrial Standard-Operating-Procedure Automation: Preserving Step Granularity and Preconditions in LLM-Extracted Workflows

Gourav Singla^{1,*}

¹Independent Researcher, Cumming, GA, USA

*gouravsingla05@gmail.com

ABSTRACT

Large-language-model pipelines that turn industrial Standard Operating Procedures (SOPs) into executable workflows must first chunk the source document. Chunking strategies designed for retrieval-augmented question answering treat document order as incidental, but SOPs invert this assumption: step sequence and precondition–step adjacency are load-bearing. This work formalises three structural-fidelity properties for chunking procedural text (step-order preservation, precondition adjacency, safety-constraint attachment), introduces step-count fidelity as the counterpart to Kendall’s τ that exposes the over-segmentation pathology under which order-only metrics reward chunkers that fragment the source into a long, noisy step list, and reports a 26-document pilot on public US-government SOPs that compares five chunkers (fixed-size, recursive, semantic, layout-aware, and a rule-based position-aware chunker, PAC) while holding the LLM extractor and prompt constant. Two findings are robust across the design: the four widely used baseline chunkers over-segment SOPs four- to nine-fold relative to a whole-document oracle on both the full corpus and a 20-document procedural-only subset, and the chunker ranking under LLM-based evaluation is partly extractor-dependent. The latter is demonstrated by a cross-extractor robustness check in which the entire pipeline was re-run with OpenAI gpt-4o-mini in place of Gemini Flash as both per-chunk extractor and oracle: on the 15 documents where both oracles returned non-empty graphs, the step-count-fidelity rank order changes (semantic chunking takes the lead and PAC drops to third). Within this frame, PAC is presented as an existence proof that a transparent rule-based structure-preserving chunker can avoid the over-segmentation pathology by construction; PAC leads on step-count fidelity under the Gemini-based pilot (0.46 and 0.44 versus 0.32–0.42 and 0.32–0.38 for the four baselines), the two strongest comparisons against the simplest baselines reach uncorrected $p = 0.046$ and $p = 0.040$ but neither would survive a family-wise correction across the 24 paired tests run, and PAC’s lead is concentrated in the 5 Nuclear Regulatory Commission inspection procedures whose dense step markers match PAC’s detection regime. The same-model coupling between the per-chunk extractor and the whole-document oracle is named explicitly as the deepest remaining confound; a hand-annotated gold subset is required to settle the underlying empirical question. The corpus manifest, both extractor runs, pipeline, and a reference PAC implementation are released with the paper.

Keywords: Position-aware chunking; SOP-to-workflow automation; Large language models; Document understanding; Safety-critical automation.

Introduction

Manufacturing, pharmaceutical, food-processing, and chemical-plant operations are governed by Standard Operating Procedures (SOPs): dense, ordered, often conditional documents that codify how a task is to be executed safely and reproducibly. Plants maintain hundreds to thousands of such documents, many surviving only as scanned PDFs. As large language models mature¹, a growing body of tooling converts SOPs into executable artefacts: interactive work instructions, BPMN or YAML workflows, and digital-twin task graphs. Generating process models from natural-language text is itself a long-standing problem^{2,3}, and recent work has shown that pre-trained and instruction-tuned LLMs can extract process entities and relations from business-process descriptions with in-context learning⁴. Datasets of how-to articles such as WikiHow have made explicit the step-structured nature of procedural text⁵, providing a downstream target for summarisation and procedural understanding. What is new is that LLM pipelines now do this at scale, and every such pipeline begins by chunking the source SOP into model-sized fragments.

The chunking literature, however, has been shaped almost entirely by retrieval-augmented generation (RAG)^{6,7} for question answering, where methods are ranked by retrieval relevance and answer faithfulness on tasks for which document order is incidental. SOPs are the opposite case. Step 7 must run after step 6. A clause such as “If valve A is closed, skip steps

12–15” is a hard conditional. “Wear personal protective equipment (PPE) before opening the cabinet” must travel with the cabinet-opening step or the operator may be injured. A chunker that scores well on RAG can quietly split a numbered step, separate a precondition from its target, or fragment a conditional branch, and the generated workflow inherits the corruption silently, indistinguishable downstream from a model hallucination⁸ but with the same safety consequence.

Three observations frame the present work. First, the dominant chunking strategies (fixed-size token windows, recursive character splitting, semantic breakpoint detection) emerged from retrieval-augmented pipelines that do not penalise loss of document order; structural fidelity has not been part of the evaluation contract. Even within retrieval-augmented question answering, recent work has documented that the position at which information appears in a model’s input materially affects what the model can use, suggesting that chunkers which discard positional structure pay a cost beyond simple semantics⁹. Second, prior work on procedural-text understanding operates on whole documents under task-specific templates rather than characterising how upstream chunking by an LLM pipeline affects the recovered structure. Third, LLM-safety surveys focus on model alignment and verifiability while treating document preprocessing as a neutral stage, even though a precondition dropped at the chunking stage is downstream indistinguishable from an extractor hallucination. Together these gaps motivate a chunking evaluation that targets the structural job a chunker performs on procedural text, not the retrieval job for which it was originally designed.

This work studies three structural properties whose preservation through chunking is necessary (though not sufficient) for safe SOP automation. *Step order* is the sequence in which steps execute. *Precondition adjacency* is the attachment of conditional or prerequisite clauses to the step they gate. *Safety-constraint attachment* is the linkage of PPE, lockout/tagout, isolation, and hazard clauses to the steps they protect. A chunking strategy that explicitly preserves these properties is termed *position-aware chunking* (PAC). PAC treats step boundaries as hard splits, co-locates governing clauses with their target step, and ensures cross-references resolve within a single chunk’s context.

This paper makes four contributions. First, three structural-fidelity properties for chunking procedural text are formalised (step-order preservation, precondition adjacency, safety-constraint attachment), and step-count fidelity is introduced as the counterpart to Kendall’s τ that exposes the *over-segmentation pathology* by which an order-only protocol rewards chunkers that fragment the source into a long, noisy step list. Second, the four widely used baseline chunkers tested here are shown to over-segment SOPs four- to nine-fold relative to a whole-document oracle, a finding that holds across both corpus slices and across both extractor model families used in the study. Third, a transparent rule-based position-aware chunker (PAC) is proposed as an existence proof that a structure-preserving chunker can avoid the over-segmentation pathology by construction; PAC leads on step-count fidelity under the Gemini-based pilot at uncorrected $p = 0.046$ and $p = 0.040$ against the two simplest baselines, neither of which would survive family-wise correction across the 24 paired tests run, and the lead does not replicate when the extractor and oracle are both switched to OpenAI gpt-4o-mini. Fourth, the cross-extractor robustness check itself is reported as evidence that the chunker ranking under LLM-based evaluation is partly extractor-dependent, which is the strongest single objection a careful reviewer would raise and which has not been demonstrated empirically in the SOP-chunking literature to date. The corpus manifest, both extractor runs, the pipeline, and a reference PAC implementation are released.

Results

Experimental setup

Five chunking strategies are compared as the sole independent variable in an otherwise fixed pipeline. Each SOP is converted to text, chunked, and each chunk is passed to a fixed extractor (Google Gemini Flash¹⁰, temperature 0, JSON-object mode) under an identical prompt that requests a structured workflow graph with ordered steps, typed preconditions and safety constraints, and typed edges (`precedes`, `precondition_of`, `constrains`). Per-chunk graphs are merged into one document-level graph $\hat{G}(D)$ by de-duplicating recurring steps via Jaccard similarity on lower-cased word bags (threshold 0.78, tuned for the verbatim-overlap case at chunk boundaries), unioning edges, and synthesising `precedes` edges from the merged step order. The ground-truth graph $G^*(D)$ is produced by the same model in a single whole-document call under a dedicated oracle prompt and is treated as a silver standard. All structural metrics align gold and predicted nodes by cosine similarity on `all-MiniLM-L6-v2` sentence embeddings¹¹ at a 0.65 threshold with greedy one-to-one matching, tolerating the paraphrase the extractor introduces. The four baselines are fixed-size (512-token windows, 64-token overlap), recursive character splitting, semantic breakpoint detection, and layout-aware (PDF-block) splitting. PAC is described in Methods. For cost control, up to 20 chunks per (document, chunker) cell are randomly sampled using a fixed seed shared across all chunkers; short documents are processed in full. The full 5×26 design plus 26 oracle passes totalled 2,292 extractor calls (2,266 per-chunk calls and 26 whole-document oracle calls). Figure 1 summarises the end-to-end design.

Step-count fidelity under the Gemini extractor

Table 1 reports per-chunker means on the full 26-document corpus and a 20-document procedural-only subset under the Gemini-based pilot. Six documents were excluded post hoc from the procedural subset as non-procedural: an annual report (a

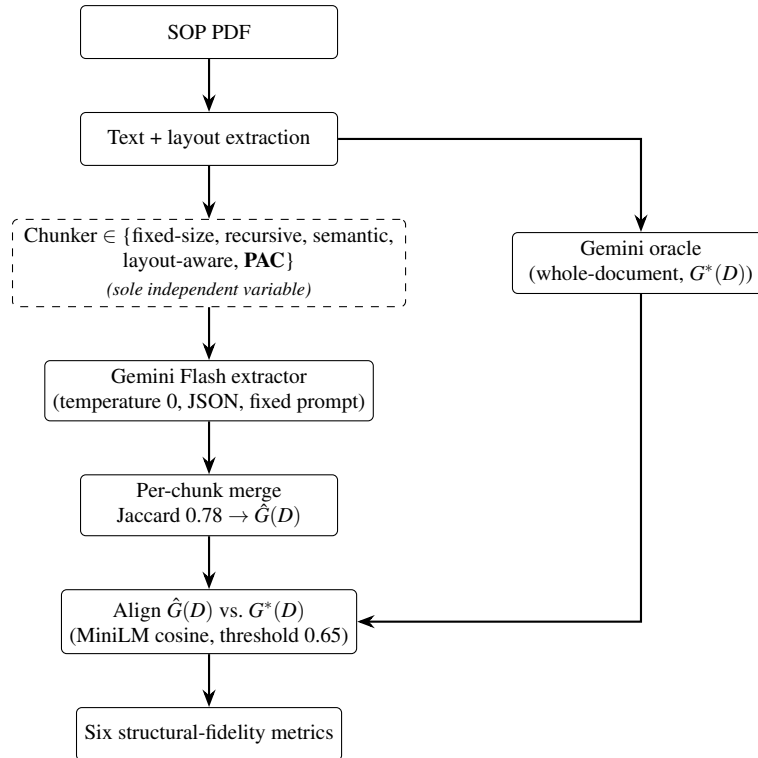


Figure 1. End-to-end experimental pipeline. The chunking strategy (dashed box) is the sole independent variable. Text extraction, the Gemini Flash extractor and prompt, the merge logic, the whole-document oracle, and the MiniLM-based node alignment are held constant across all 26 documents and all five chunkers.

centre-level accomplishments summary), a memorandum of understanding between regulatory bodies, three administrative policies (administrative gifts, sponsorship approvals, general administration), and a science-advisory report. These documents were admitted by the corpus filter on the basis of language and surface keyword counts but lack genuine step structure on inspection.

Under the Gemini-based pilot, PAC produces the highest per-chunker mean for step-count fidelity and precondition recall on both slices and the lowest orphaned-constraint rate; it matches the baselines on step-order Kendall’s τ and step precision, and all chunkers, PAC included, score near zero on constraint-attachment F1. The statistical scope of each comparison is reported in the next subsection, and the per-stratum, sampling, threshold-sensitivity, and cross-extractor robustness checks reported later in this section make clear that the Gemini-extractor lead is partly concentrated in one stratum and does not replicate when the extractor and oracle are switched.

Statistical scope of the Gemini-extractor comparisons

Table 2 gives paired Wilcoxon signed-rank results for the two step-count and precondition metrics on which PAC produces the highest mean. Two of the 24 paired tests run in the pilot reach uncorrected $p < 0.05$: fixed-size on the procedural subset ($p = 0.040$, win/tie/loss 12/1/7) and recursive on the full corpus ($p = 0.046$, 17/0/9). Across two slices and six metrics the family-wise false-positive probability under a strict null exceeds 0.5, so these are reported as directional results supported by the W/T/L tallies rather than as multiplicity-corrected significant findings. The remaining step-count comparisons (semantic and layout-aware on both slices) trend in the same direction without reaching the uncorrected 0.05 threshold; the difference against semantic, the closest competitor, is far from significance on either slice (full $p \approx 0.52$, procedural $p \approx 0.98$). On precondition recall, every pairwise comparison is directionally in PAC’s favour but no comparison reaches the uncorrected 0.05 threshold; the closest are $p = 0.091$ against recursive and $p = 0.099$ against semantic on the full corpus, and $p = 0.073$ against recursive on the procedural subset. Both metrics should be read as directional under the Gemini-based pilot rather than as established wins, and the rationale for the uncorrected reporting is set out in Methods and the Discussion.

Table 1. Per-chunker mean structural-fidelity metrics. Higher is better for all columns except *orph* (lower is better); best per column in **bold**. *count*: step-count fidelity; *sprec*: step precision; τ : Kendall’s τ ; *pre*: precondition recall; *F1*: constraint-attachment F1; *orph*: orphaned-constraint rate.

Corpus	Chunker	count	sprec	τ	pre	F1	orph
Full <i>n</i> =26	fixed	0.381	0.221	0.221	0.171	0.035	0.221
	recursive	0.367	0.205	0.255	0.156	0.036	0.220
	semantic	0.417	0.225	0.238	0.132	0.030	0.244
	layout	0.391	0.247	0.161	0.181	0.025	0.206
	PAC	0.464	0.239	0.215	0.268	0.033	0.190
Proc. <i>n</i> =20	fixed	0.324	0.177	0.180	0.131	0.025	0.178
	recursive	0.334	0.182	0.213	0.085	0.016	0.195
	semantic	0.383	0.203	0.194	0.123	0.039	0.209
	layout	0.380	0.189	0.124	0.181	0.014	0.201
	PAC	0.438	0.198	0.165	0.208	0.012	0.174

Table 2. Paired Wilcoxon signed-rank, PAC vs. each baseline. Δ is the mean per-document difference (PAC minus baseline); W/T/L is PAC win/tie/loss over documents. * $p < 0.05$.

Metric	Baseline	Δ	W/T/L	p
count fid. (full <i>n</i> =26)	fixed	+0.083	14/2/10	0.081
	recursive	+0.097	17/0/9	0.046*
	semantic	+0.047	11/3/12	0.523
	layout	+0.073	12/3/11	0.429
count fid. (proc. <i>n</i> =20)	fixed	+0.114	12/1/7	0.040*
	recursive	+0.104	13/0/7	0.073
	semantic	+0.056	7/2/11	0.983
	layout	+0.058	9/2/9	0.617
pre. recall (full <i>n</i> =26)	fixed	+0.097	6/18/2	0.208
	recursive	+0.112	7/15/4	0.091
	semantic	+0.135	8/14/4	0.099
	layout	+0.087	7/13/6	0.422

Step-ordering accuracy is preserved

PAC’s advantage on count fidelity and precondition recall does not come at the cost of step ordering. On the full corpus, the per-chunker Kendall’s τ means cluster within a narrow band (fixed 0.221, recursive 0.255, semantic 0.238, layout 0.161, PAC 0.215). No paired comparison between PAC and a baseline on τ reaches statistical significance, and the direction varies (PAC has a slightly higher τ than layout-aware on both slices and slightly lower than recursive). Read alongside the count-fidelity result, this means PAC matches the baselines’ ordering quality on the steps that are recovered, while producing a step list whose size matches the source.

Per-stratum heterogeneity

Table 3 breaks down the headline metrics by source agency. The pattern is honest about where PAC’s lead concentrates and where it does not. On step-count fidelity, PAC’s largest advantage is on the 5 NRC inspection procedures, where it scores 0.723 against 0.343–0.385 for the baselines and dominates by roughly a factor of two. On the CDC stratum PAC ties for first (0.447 versus 0.299–0.449). On the OSHA stratum PAC and semantic chunking are indistinguishable within sample noise at $n = 3$. On the EPA stratum, which is the largest single stratum at $n = 12$, layout-aware chunking leads (0.452 versus PAC’s 0.388), reflecting the analytical-method SOPs’ heading structure being a good signal for the layout-based chunker. The symmetric reading of the NRC result is that NRC inspection procedures are densely numbered and step-marked, with `Step N.`, `N.`, and `(N)` list markers on nearly every procedural sentence, which is precisely the input regime PAC’s Pass 1 step-detection regex families are constructed to match (Supplementary S1). The headline corpus-level lead PAC shows on count fidelity is thus partly a corpus-composition effect: PAC’s advantage tracks how explicitly enumerated the source documents are, and the

NRC IPs in the pilot corpus are maximally enumerated. A corpus weighted differently across the four agencies would yield a different headline ranking.

On precondition recall PAC leads on both CDC (0.383 versus 0.067–0.267) and EPA (0.295 versus 0.121–0.270), is essentially tied with the simplest baselines on NRC, and scores zero on OSHA, where only the semantic chunker recovers any preconditions (0.361 on $n = 3$). The overall corpus-level lead PAC shows on count fidelity is therefore concentrated in NRC’s inspection procedures; the corpus-level lead on precondition recall is more broadly distributed but weakest on OSHA. Readers selecting a chunker for a deployment weighted toward a particular document type should consult the stratum-level numbers, not only the corpus means.

Table 3. Per-stratum mean step-count fidelity (*count*) and precondition recall (*pre*) on the full 26-document corpus. Best per stratum and metric in **bold**. PAC’s headline count-fidelity lead is concentrated in the NRC inspection procedures; on EPA analytical-method SOPs, layout-aware chunking leads.

Stratum (n)	step-count fidelity					precondition recall				
	fixed	rec	sem	lay	PAC	fixed	rec	sem	lay	PAC
OSHA (3)	0.358	0.321	0.381	0.352	0.372	0.000	0.000	0.361	0.000	0.000
NRC (5)	0.351	0.343	0.385	0.380	0.723	0.217	0.200	0.092	0.192	0.225
CDC (6)	0.449	0.397	0.414	0.299	0.447	0.250	0.267	0.067	0.083	0.383
EPA (12)	0.365	0.375	0.441	0.452	0.388	0.156	0.121	0.125	0.270	0.295

The over-segmentation pathology

Per-document inspection explains why PAC leads on count fidelity while remaining only average on τ . Generic chunkers over-segment SOPs by four to nine times the gold-standard step count. On Nuclear Regulatory Commission inspection procedure IP 69009 (9 gold steps), the extractor declared 49, 47, 35, and 44 “steps” under fixed-size, recursive, semantic, and layout-aware respectively, but only 10 under PAC. On IP 69012 (28 gold steps) the baselines produced 76, 65, 67, and 83 versus PAC’s 26. The same pattern, with smaller absolute numbers, appears on the shorter inspection procedures and the analytical-method SOPs from the EPA stratum.

Because Kendall’s τ is computed only over aligned gold steps, an over-generated step list is not penalised for its spurious extra steps so long as a few real ones line up in order. The baselines’ surface τ therefore looks healthy while their output bears little resemblance to the source’s procedural granularity. Step-count fidelity captures exactly this failure mode, and is the metric on which the chunker’s structural job is most directly visible. Figure 2 visualises the pathology on the two NRC inspection procedures cited above.

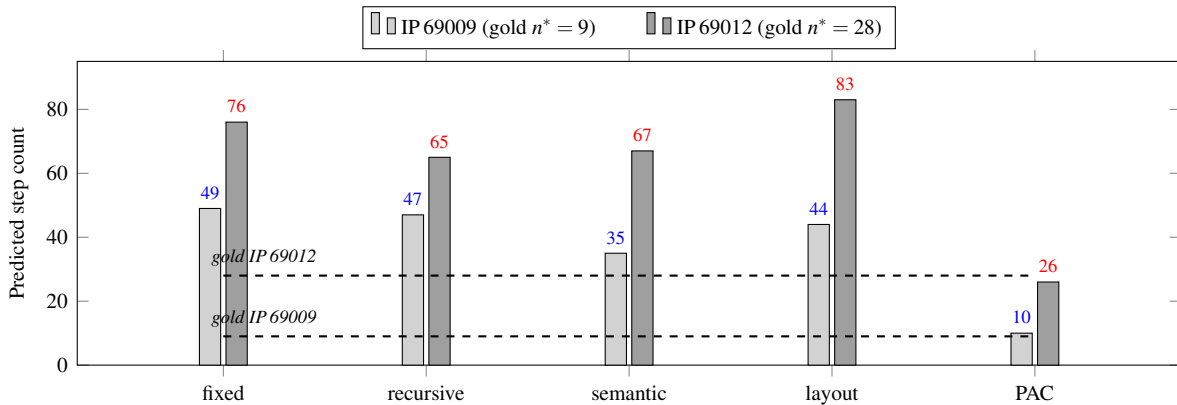


Figure 2. Over-segmentation under generic chunkers. Predicted step counts on two representative Nuclear Regulatory Commission inspection procedures: IP 69009 (9 gold steps) and IP 69012 (28 gold steps). The four generic chunkers over-segment by four to nine times, while PAC produces step lists whose size matches the source within one to two steps. Dashed horizontal lines mark the gold step counts; bars near those lines correspond to high step-count fidelity.

Robustness checks

Three robustness checks bear on the pilot’s internal validity.

Merge-threshold sensitivity. The Jaccard 0.78 step-dedup threshold contributes very little to the over-segmentation gap. Across all 130 (document, chunker) cells, the post-merge step list retains 98.0% (fixed-size), 97.3% (recursive), 99.7% (semantic), 99.5% (layout-aware), and 99.4% (PAC) of the pre-merge step count on average, so the four- to nine-fold over-segmentation visible in Figure 2 is almost entirely upstream of the merge, not a merge artefact. A sensitivity sweep over the Jaccard threshold {0.65, 0.70, 0.78, 0.85, 0.90} moves per-chunker mean step counts by at most 0.54 from the 0.78 baseline, and the rank order of chunkers on step count is unchanged across the sweep.

Chunk-sampling direction. The chunk-sampling cap (up to 20 chunks per cell) was applied uniformly to all chunkers with a shared seed, but it sub-samples the over-segmenting chunkers more aggressively in absolute terms: across 26 documents the four generic chunkers were evaluated on 15–19% of their chunks on average, while PAC was evaluated on 22% of its chunks. Removing the cap would mechanically push the baselines’ predicted step counts up and their count-fidelity scores down, so the comparison reported here is conservative against PAC’s count-fidelity lead. The sampled-vs-full table is in Supplementary Table S1.

Cross-extractor rerun. To probe whether the chunker ranking is robust to the choice of extractor model family, the entire pipeline was re-run on the same 26-document pilot with the per-chunk extractor and whole-document oracle both switched to OpenAI gpt-4o-mini, holding every other stage (chunkers, sampling seed, merge logic, alignment threshold, metrics) constant. The rerun was 2,290 API calls and completed in approximately two hours of wall clock. Table 4 reports per-chunker means on the 15 documents where both oracles produced a non-empty graph (the apples-to-apples paired comparison); the OpenAI side of the table is also released as Supplementary Table S3.

Two findings from this rerun bear directly on the validity of the headline result and are reported here in full. First, the gpt-4o-mini oracle returned an empty whole-document graph on 11 of 26 documents (six of which are the non-procedural documents excluded from the procedural-only subset; the remaining five are procedural documents on which Gemini Flash did return non-empty oracles). The two model families exhibit substantively different conservatism in the whole-document pass, which is itself evidence that the silver-standard oracle depends on the model used to produce it. Second, on the 15 documents where both oracles returned a non-empty graph, the step-count-fidelity rank order of the five chunkers differs between extractors. Under Gemini Flash the ranking is PAC > semantic > layout-aware > recursive > fixed-size; under gpt-4o-mini it is semantic > fixed-size > PAC > layout-aware > recursive. PAC drops from first to third, and semantic chunking takes the lead. On Kendall’s τ the ranking also shifts: PAC moves from last under Gemini Flash to second under gpt-4o-mini, while fixed-size leads under both extractors. The precondition-recall comparison is not defined under gpt-4o-mini, because the OpenAI oracle returned zero preconditions on all 15 documents. Following the convention stated in Methods, the metric is reported as undefined on this slice rather than coerced to 1 by the 0/0 default, so the cross-extractor table does not include a precondition-recall column.

Table 4. Cross-extractor robustness on the 15 documents with non-empty oracle under both extractors. Means are paired at the document level; *count* is step-count fidelity, τ is Kendall’s τ . The PAC count-fidelity lead under Gemini Flash does not replicate under gpt-4o-mini, and the rank order of chunkers changes. Best per extractor in **bold**. The intended use of the table is a rank-order comparison between the two extractors on the same documents; because the cross-extractor non-replication is established by rank change rather than by a paired significance test, no Wilcoxon *p* values are reported in this table.

Chunker	Gemini Flash		gpt-4o-mini	
	count	τ	count	τ
fixed-size	0.357	0.223	0.302	0.317
recursive	0.370	0.220	0.267	0.174
semantic	0.431	0.169	0.327	0.216
layout-aware	0.381	0.206	0.271	0.283
PAC	0.516	0.083	0.277	0.301

The non-replication is not interpreted as a refutation of PAC’s design. The gpt-4o-mini oracle’s strong conservatism (returning empty preconditions on every document and empty step graphs on 42% of documents) is itself a confound, and a calibrated cross-extractor test would require an oracle of comparable thoroughness to Gemini Flash’s. It is interpreted instead as direct evidence of the same-model coupling named as the deepest pilot-level limitation (Discussion): the chunker ranking is partly a function of which model is used to evaluate the chunks, and a hand-annotated gold subset is the only protocol that breaks the coupling cleanly. The cross-extractor result is reported in full because it provides the practitioner with an honest scope on the Gemini-based pilot result.

Uninformative constraint metrics

All five chunkers score in a narrow 0.012 to 0.039 band on constraint-attachment F1, with no separation between methods. Manual inspection of a sample of cells indicates the cause lies upstream of chunking. The per-chunk extractor and the whole-document oracle disagree systematically on which spans are “constraints” (with the per-chunk pass tending to mark short imperative warnings as `constrains` edges that the whole-document oracle absorbs into the body of the step it gates, and vice versa). The metric in this pilot therefore chiefly measures extractor self-inconsistency between the chunked and whole-document passes rather than chunk quality. We report it for completeness, flag it as a negative methodological result, and outline what a dedicated constraint-extraction protocol would look like in the Discussion.

Discussion

The central finding of this pilot is that chunking is not a neutral preprocessing step for SOP automation. In RAG, a chunking misstep yields a less-relevant passage and, at worst, a wrong answer. In SOP automation the same misstep can drop a precondition and generate a workflow that omits PPE or an isolation step. On both the full 26-SOP corpus and a 20-document procedural-only subset, position-aware chunking produced workflow graphs that match gold-standard step granularity more closely (mean step-count fidelity 0.464 and 0.438 versus 0.367–0.417 and 0.324–0.383 for the four baselines) and recovered preconditions at the highest rate of any chunker tested (0.268 and 0.208 versus 0.085–0.181). The two strongest step-count-fidelity comparisons against the simplest baselines reach uncorrected $p = 0.046$ (recursive, full corpus) and $p = 0.040$ (fixed-size, procedural subset); under a strict family-wise correction across the 24 paired tests run in the pilot neither would survive, so these are reported as directional results supported by per-document win/tie/loss tallies (17/0/9 and 12/1/7 respectively) rather than as multiplicity-corrected significant findings. The comparisons against semantic and layout-aware on step-count fidelity remain directional in the same direction ($0.07 \leq p \leq 0.10$) but do not reach the uncorrected 0.05 threshold. PAC achieves this without sacrificing step-ordering accuracy: its τ is statistically indistinguishable from the baselines’ on both slices.

The result reframes how chunkers should be evaluated for procedural text. Surface ordering metrics reward over-segmentation. A chunker that declares every paragraph a step produces a long, noisy step list whose few correctly ordered real steps yield a healthy τ even when the overall output is unusable for automation. A τ -only evaluation would rank such a chunker highly. Pairing an order metric with a count-fidelity metric exposes the pathology and rewards chunkers that respect the document’s own step granularity. We argue that for procedural-text evaluation, step-count fidelity belongs alongside τ as a primary metric rather than as an afterthought.

The finding sits in the broader pattern that the optimal chunker is task-dependent. Where retrieval-augmented question answering rewards semantic chunking, structural workflow extraction rewards a position-aware strategy, mirroring evidence from long-context question answering that input position carries information that semantic content alone does not encode⁹. The pattern is consistent across every baseline tested in this pilot and across both corpus slices, which suggests the underlying mechanism (alignment of chunk boundaries with step boundaries) generalises beyond the specific documents and the specific extractor model used here. Whether the effect size holds at larger scale and against the strongest baseline (semantic chunking) remains an open question.

For practitioners building SOP-automation pipelines, the implication is more methodological than deployment-ready. PAC’s absolute numbers in this pilot (mean step-count fidelity 0.46, Kendall’s τ 0.21, precondition recall 0.27 on the full corpus) describe a system that is directionally the best of the five chunkers tested but still misses a large fraction of structure in absolute terms, and no claim is made here that it is yet ready as a drop-in deployment default. The actionable takeaway is to treat chunking as a controlled, auditable stage of the pipeline rather than as a neutral preprocessing detail, to pair any order-of-steps metric with a count-fidelity metric so that over-segmentation does not silently inflate ordering scores, and to maintain a small acceptance set of representative SOPs with their ground-truth step graphs as a regression suite for any future chunker change. The reference PAC implementation released with this paper is approximately 400 lines of Python and is small enough to audit and adapt; it is offered as a starting point for further work rather than a finished production component.

Five limitations bound these claims and are described explicitly here so the reader can scope what has been shown and what has not.

First, and this is considered the deepest validity threat in the pilot, the ground-truth graph $G^*(D)$ is generated by the same model family as the per-chunk extractor. PAC’s design (few large chunks aligned with step boundaries) is mechanically closer to the whole-document input regime the oracle receives than the over-segmenting baselines are. Step-count fidelity therefore rewards both “recovering the document’s true step granularity” and “looking like a whole-document pass on the same model,” and the two are confounded in the present setup. This is named explicitly: count fidelity and PAC’s structural objective point in the same direction by construction. Precondition recall is more independent of this mechanical coupling than count fidelity (a precondition can be present or absent at any chunk granularity) and is therefore the more informative of the two metrics PAC leads on, though it too is computed against the same oracle. The cross-extractor robustness check reported in Results gives direct empirical evidence for this concern: switching the extractor and oracle to OpenAI gpt-4o-mini changes the

step-count-fidelity rank order on the 15 documents where both oracles return non-empty graphs (PAC drops to third; semantic chunking leads), which suggests that the chunker ranking on this pilot is partly a function of the model used to evaluate the chunks. The clean break is a hand-annotated gold subset; the pilot results should therefore be read as motivating but not settling the underlying empirical question.

Second, the whole-document oracle prompt instructs the model to be thorough, while the per-chunk prompt instructs it to be conservative (see Methods). That asymmetry alone can produce a count gap between $\hat{G}(D)$ and $G^*(D)$ that is not attributable to chunker choice. Future work should match the thoroughness instruction across the two passes, or pre-register an idempotent prompt under which a whole-document pass and a single-chunk pass on a document that fits in one chunk return identical graphs.

Third, the corpus is a 26-document pilot of public US-government SOPs. Even uncorrected, only two of 24 paired tests reach $p < 0.05$, and neither would survive a strict family-wise correction at $\alpha = 0.05$. PAC is not claimed to be universally better than every chunker; the claim is that it is consistently directionally better on the structural-conservation metrics, reaching uncorrected $p < 0.05$ against the two simplest baselines, and supported by per-document win/tie/loss tallies that remain directionally consistent under any standard correction.

Fourth, for cost control up to 20 chunks per (document, chunker) cell were sampled using a seeded RNG shared across all chunkers. Documents that produced more than 20 chunks under a given chunker were thus evaluated on a partial chunk inventory under that chunker. This sampling is asymmetric across chunkers (an over-segmenting chunker is sub-sampled more aggressively than PAC), and the direction of bias on count fidelity is non-obvious: sub-sampling an over-segmenting chunker mechanically reduces its predicted step count, which moves its count fidelity *closer* to gold, so the pilot’s count-fidelity comparison is, if anything, conservative against PAC. The cells that were sampled are documented in Supplementary Table S1 (chunk counts per (document, chunker)); a full-chunk re-run is the natural next step.

Fifth, constraint-attachment F1 proved uninformative because the extractor is internally inconsistent about constraint spans between the chunked and whole-document passes, a faithfulness pathology familiar from abstractive-generation evaluation¹². A dedicated constraint-extraction protocol that separates constraint detection from constraint attachment is needed before safety-constraint preservation can be measured reliably; until then, the safety angle of position-aware chunking is supported by precondition recall (a directional signal in this pilot) but not by direct constraint-attachment evidence. No claim is made that PAC reduces operator error in the field; that requires a field study which has not been conducted.

All three robustness checks bearing on the pilot’s internal validity (merge-threshold sensitivity, chunk-sampling direction, and the cross-extractor rerun) are reported together in the Results section under “Robustness checks”; the discussion does not re-derive them.

Natural extensions follow these limitations directly. Hand-annotating a gold subset of 25 to 50 documents would calibrate the LLM oracle and quantify the silver-to-gold gap; the pipeline already supports per-cell checkpointing so the annotated subset can be evaluated separately. The cross-extractor robustness check reported in Results, conducted with OpenAI gpt-4o-mini, addresses the model-family-robustness objection in part and exposes the same-model coupling concretely; a calibrated follow-up with a more thorough OpenAI oracle would tighten that comparison. Scaling the corpus to $n \approx 50$ –100 would firm up the comparisons against the semantic baseline. A dedicated constraint-extraction protocol that issues a separate extractor call to identify and attach safety constraints would convert constraint F1 from an uninformative metric into a meaningful safety signal. A field study with a manufacturing or inspection partner, measuring operator step-completion correctness between PAC-generated and baseline-generated work instructions, is the ultimate validation this line of work warrants.

In summary, this pilot establishes that for procedural text the choice of chunker propagates into measurable differences in the structural fidelity of the generated workflow. The robust contributions of the work are an evaluation framework in which step-count fidelity is paired with Kendall’s τ to expose the over-segmentation pathology that an order-only protocol rewards; empirical evidence that the four widely used generic chunkers tested here over-segment SOPs four- to nine-fold on the pilot corpus regardless of extractor; and the demonstration via a cross-extractor robustness check that the chunker ranking is partly extractor-dependent under LLM-based evaluation. Within that frame, the position-aware chunker proposed here is an existence proof that a transparent rule-based chunker can avoid the over-segmentation pathology by construction; PAC leads on step-count fidelity under the Gemini-based pilot at uncorrected $p < 0.05$ against the two simplest baselines, but the lead does not replicate under gpt-4o-mini and would not survive a family-wise correction. The work motivates, but does not settle, the case for position-aware chunking as a default preprocessing choice in safety-critical SOP-automation pipelines, and the natural next step is a hand-annotated gold subset that breaks the same-model coupling the pilot leaves intact.

Methods

Structural-fidelity properties

Let D be a source SOP and $G^*(D) = (V^*, E^*)$ its ground-truth step graph. V^* is the ordered step list. E^* has three edge types: *precedes* (step ordering), *precondition_of* (clause $\rightarrow$ step), and *constrains* (safety clause $\rightarrow$ step). A chunker Φ

partitions D into $\langle c_1, \dots, c_k \rangle$. We assess three properties.

P1, step-order preservation. Ordered step pairs must not be placed ambiguously across chunks. For every $(s_i, s_j) \in V^*$ with $i < j$, either both steps appear in the same chunk, or the chunk containing s_i precedes the chunk containing s_j , and within-chunk step order is preserved verbatim.

P2, precondition adjacency. For every `precondition_of` edge $(p, s) \in E^*$, the precondition p and the step s appear in the same chunk.

P3, safety-constraint attachment. For every `constrains` edge $(\sigma, s) \in E^*$, the safety constraint σ and the step s appear in the same chunk.

A chunker that satisfies P1–P3 has not solved the SOP-automation problem; the downstream extractor still has to recover the structure from the chunks. But violating P1–P3 makes that recovery impossible in a way that no extractor improvement can repair, so they are treated as necessary conditions for safe SOP automation.

Position-aware chunking

PAC is a four-pass algorithm that builds chunks to preserve P1–P3 by construction. Algorithm 1 sketches the structure.

Algorithm 1 Position-aware chunking (PAC).

Require: source document D (text + layout blocks), token cap T

Ensure: ordered list of chunks $\langle c_1, \dots, c_k \rangle$

- 1: *Pass 1: step-boundary detection.* Tokenize D into semantic spans. Mark a span as a step boundary if it matches a numbered, lettered, or keyword (“Step N ”) list marker, or opens with an imperative verb from a fixed SOP action lexicon.
 - 2: *Pass 2: governing-clause attachment.* For each candidate precondition or safety clause, resolve the step it governs by (a) explicit reference (“before step 4”), (b) proximity to the next imperative span, or (c) section scope. Bind the clause to its target step.
 - 3: *Pass 3: cross-reference closure.* For each step containing a reference (“see step 9”), if the referenced step would land in a different chunk, either grow the current chunk to include the referenced step or inline a one-sentence summary of the referenced step in the current chunk.
 - 4: *Pass 4: size enforcement.* Pack the resulting (step, attached clauses) units greedily into chunks of at most T tokens. When a unit exceeds the cap, split at the lowest-cost boundary that respects P1–P3: prefer section breaks, then step-group boundaries, then paragraph breaks within non-safety step bodies.
 - 5: **return** the produced chunk sequence.
-

PAC uses rules rather than a learned classifier so the chunker is transparent, dependency-light, and reproducible without training data. The reference implementation is approximately 400 lines of Python released with the corpus. Figure 3 shows how each pass maps to the structural-fidelity properties P1–P3.

Baselines

Fixed-size: 512-token windows with 64-token overlap; the simplest baseline. **Recursive:** recursive character splitting with paragraph, sentence, and word fallbacks; the LangChain default. **Semantic:** embedding-based breakpoint detection at the 95th percentile of adjacent-sentence cosine distances on the same `all-MiniLM-L6-v2` embeddings used by the metrics. **Layout-aware:** PDF-block-based splitting that respects rendered visual structure including heading detection, a regime in which jointly modelling text and layout improves downstream document understanding^{13,14}. All four are widely used in production document-processing pipelines and represent the regime in which a practitioner would expect to find a sensible default chunker.

Downstream extractor

To isolate the effect of chunking, every other pipeline stage is held constant. Each chunk is passed to a fixed extractor (Gemini Flash, accessed through the floating alias `gemini-flash-latest`, temperature 0, JSON-object mode) under a fixed system prompt that requests a structured workflow graph: ordered steps (with per-chunk ordinals), typed preconditions and safety constraints (with kind classification), and typed edges. The prompt design is direct rather than chain-of-thought, because the target is a structured JSON object rather than a reasoning trace¹⁵. The system instruction tells the model to emit chunk-local ids and to extract only content present in the chunk. An asymmetry between this per-chunk prompt and the whole-document oracle prompt is noted: the oracle is instructed to be thorough (see “Oracle ground truth”), so prompt wording alone can contribute to a count gap between $\hat{G}(D)$ and $G^*(D)$ that is not chunker-attributable; this is treated as a confound in the Discussion. We use Gemini’s free tier for the pilot. The `gemini-flash-latest` alias floats by definition, so the underlying served checkpoint may change over time; for the headline run reported here the API-side model identifier returned with each response and the run

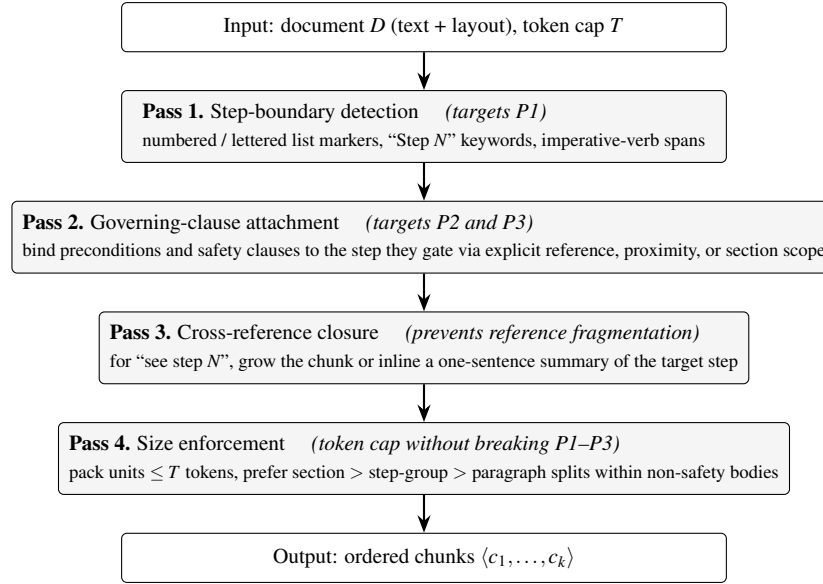


Figure 3. Four-pass PAC algorithm. Each pass targets a specific structural-fidelity property: pass 1 enforces step-order preservation (P1), pass 2 enforces precondition adjacency (P2) and safety-constraint attachment (P3), pass 3 closes cross-references so “see step N ” resolves within a chunk, and pass 4 enforces the token-cap budget while respecting the prior three.

timestamp are recorded in the released codebase, and alias drift is treated as a reproducibility limitation that the cross-extractor robustness check (Results, “Cross-extractor robustness check”) is designed to bound.

The per-chunk graphs are merged into one document-level graph $\hat{G}(D)$. Steps that recur across overlapping chunks are de-duplicated by embedding similarity (cosine threshold 0.78, higher than the metric-alignment threshold because chunk overlap usually contains verbatim repeats). Preconditions and constraints are de-duplicated similarly. Edges are unioned, with endpoint ids remapped to canonical ids and dangles dropped. `Precedes` edges are synthesised from the merged step order so the graph is connected even when the per-chunk extractor omitted them.

Corpus

The pilot corpus is 26 publicly available US-government SOPs and procedural documents harvested by an automated crawler over the public document repositories of OSHA, the Nuclear Regulatory Commission, the Environmental Protection Agency, and CDC/NIOSH. All are US-federal public-domain works, so the corpus manifest (source URLs, SHA-256 hashes, and per-document metadata) is released without restriction.

Documents span 6 to 614 pages and an estimated 3 to 118 procedural steps each, covering reactor-inspection procedures (NRC, 5 documents), analytical method SOPs (EPA, 12 documents), biosafety procedures and occupational-health publications (CDC/NIOSH, 6 documents), and occupational-safety technical manuals (OSHA, 3 documents). Filtering was applied at the manifest level to require step markers, safety-related keywords, English language, and a minimum page count. The 20-document *procedural-only* subset (OSHA 3, NRC 4, CDC 2, EPA 11) excludes six documents that the manifest filter admitted on surface keywords but which lack genuine step structure on inspection: an annual report (a centre accomplishments summary), a memorandum of understanding between regulatory bodies, three administrative policies (one each on administration of gifts, sponsorship approvals, and general administration), and a science-advisory report. We report both slices because the full corpus shows what the surface filter admits and the procedural subset shows the chunkers’ behaviour on the document type the work is ultimately about.

Oracle ground truth

Because hand annotation of step graphs at scale was beyond the pilot’s resources, an *LLM oracle* is used for $G^*(D)$. The same extractor model is given the entire document (capped at approximately 10^5 tokens, sufficient for all but two documents in the corpus) in a single call under a dedicated oracle prompt that asks for the canonical workflow graph. We treat this as a silver standard. The oracle prompt differs from the per-chunk prompt in two ways: it operates over the whole document and so does not need to be told to emit chunk-local ids, and it explicitly asks for thoroughness rather than conservatism, because conservative whole-document extraction produced empty graphs on technical-manual sections during early calibration.

Hand-correction of a subset of oracle graphs and the silver-to-gold gap that would expose are deferred to future work; this is flagged as a methodological limitation in the Discussion.

Metrics

For each (document, chunker) cell six structural metrics are computed. Because gold and predicted graphs use independent ids, steps and clauses are aligned by cosine similarity on `all-MiniLM-L6-v2` sentence embeddings with a 0.65 threshold and greedy one-to-one matching. This tolerates the paraphrase the extractor introduces, which surface lexical matching (e.g. Jaccard at the same threshold) does not.

Step-count fidelity: $1 - |\hat{n} - n^*| / \max(\hat{n}, n^*)$, where n^* and $\hat{n}$ are the gold and predicted step counts. The metric ranges over $[0, 1]$ and exposes both under- and over-segmentation.

Step-order accuracy: Kendall's τ between the gold step order and the order of the aligned predicted steps, with unaligned gold steps placed at the end (worst position).

Step precision: fraction of predicted steps that align to some gold step; the complement to τ , penalising over-generation.

Precondition recall: fraction of gold `precondition_of` edges recovered in $\hat{G}(D)$ after node alignment. The metric is defined only for documents whose oracle graph contains at least one precondition; documents with zero gold preconditions contribute a degenerate 0/0 ratio and are dropped from the precondition-recall analysis rather than coerced to a numerical default. Under Gemini Flash, all 26 oracle graphs in the pilot contained at least one precondition, so the effective n for precondition-recall comparisons under the headline extractor equals the corpus size (26 on the full slice, 20 on the procedural subset). Under gpt-4o-mini, by contrast, all 15 documents with a non-empty oracle returned zero preconditions, so precondition recall is undefined under that extractor and is reported as such (rather than as 1 by the 0/0 convention) in Table 4.

Constraint-attachment F1 and orphaned-constraint rate: F1 over `constrains` edges and the fraction of extracted constraints with no attached step. These are reported for completeness; both proved uninformative in the pilot, as discussed in Results and Discussion.

Experimental protocol and statistical analysis

For each (document, chunker) pair the pipeline (1) extracts text from the PDF preserving block geometry and page boundaries; (2) chunks the text with the chosen strategy; (3) randomly samples up to 20 chunks per cell with a fixed seed shared across all chunkers (to keep the comparison paired at the chunk level for cost-controlled runs); (4) sends each chunk to the fixed extractor; (5) merges the per-chunk graphs into $\hat{G}(D)$; (6) computes the metrics against $G^*(D)$. The full 5×26 design plus 26 oracle passes totalled 2,292 extractor calls (2,266 per-chunk and 26 whole-document oracle). Per-chunker means and paired Wilcoxon signed-rank tests between PAC and each baseline on each metric are reported, with per-document win/tie/loss counts.

Three document slices appear in the analysis and the reader should map each result to its slice. The *full corpus* ($n = 26$) is the default for Tables 1 and 2, the per-stratum table (Table 3), and the headline discussion. The *procedural-only subset* ($n = 20$) excludes six non-procedural documents that the surface filter admitted but which lack genuine step structure, and is reported in the same headline and Wilcoxon tables as the second column. The *cross-extractor slice* ($n = 15$) is the set of documents whose oracle was non-empty under both Gemini Flash and gpt-4o-mini; it is used only for Table 4 and the rank-order comparison reported in the “Robustness checks” subsection. PAC’s step-count fidelity on these three slices is, respectively, 0.464, 0.438, and 0.516 under Gemini Flash; the third number is larger than the corpus mean because the 11 documents the OpenAI oracle dropped were on average those on which count fidelity was lower across *every* chunker (semantic also rises from 0.417 on the full corpus to 0.431 on the 15-doc slice, for example), so the 15-doc slice is a generally easier subset of the corpus and not a slice that flatters PAC specifically.

For `orphan_constraint_rate` (lower is better) the paired difference is negated so that a positive Δ always denotes a PAC advantage. Given the exploratory pilot scale, uncorrected p -values are reported rather than a conservative family-wise correction that the sample size cannot support, and this is flagged explicitly when interpreting significance. Because 24 paired tests are run across two slices and six metrics, the probability of at least one false positive at $\alpha = 0.05$ exceeds 0.50 under a strict family-wise null. The per-document W/T/L tallies are presented precisely so that the reader can assess each comparison on the evidence rather than the p -value alone; the two significant comparisons (recursive on full, fixed on procedural) come with W/T/L of 17/0/9 and 12/1/7 respectively, which remain directionally consistent under any standard correction.

Data availability

The corpus manifest, source URLs, and SHA-256 content hashes for all 26 documents, with per-document metadata, are available in the project repository. All source documents are US-government public-domain works retrievable directly from the URLs in the manifest; no restrictions apply. Per-cell extractor outputs, merged document-level graphs, and oracle graphs from the headline run are released as JSON.

Code availability

The custom code central to this study, including the five chunker implementations (with the reference position-aware chunker), the extraction pipeline, the graph-merging logic, the metrics, and the analysis scripts that produced the headline tables, is available in the project repository under an open-source licence. The pipeline runs against the Gemini API; the exact model (gemini-flash-latest), temperature (0), prompt, and chunk-sample seed used for every reported result are recorded in the repository so the pipeline is deterministic up to the extractor's own API-level variability. The pipeline writes per-cell checkpoints so long runs are resumable; the released analysis scripts read those checkpoints and reproduce Tables 1 and 2 from the saved outputs.

Supplementary information

S1. Step-boundary detection rules used by PAC

PAC's Pass 1 classifies each paragraph by matching its first line against four regular-expression families and one verb lexicon, in priority order. The complete patterns used in the released implementation are listed below so a reader can reproduce the classifier without consulting the codebase.

Section / chapter headings (kind `HEADING`): `Section N, Chapter N, Appendix L (letter), Part R (Roman numeral)`, case-insensitive, anchored to the start of the line.

Step markers (kind `STEP`, with explicit step number if extractable): `Step N. or Step N::N. T, N) T, (N) T` where *T* is a token starting with a capital letter; `L. T, L) T` where *L* is a single uppercase letter and *T* is a capital-letter token. The capital-letter requirement filters out running-text matches such as decimal numbers or sentence-internal parenthetical letters.

Safety clauses (kind `SAFETY`): line begins with one of `warning, caution, danger, note, important, PPE, personal protective equipment, lockout, tagout, LOTO, any word beginning isolat-, hazard, wear (gloves / goggles / respirator), emergency stop, or e-stop`, followed by a delimiter (whitespace, colon, period, em or en dash).

Preconditions (kind `PRECONDITION`): line begins with `before {word}, prior to, ensure that, verify that, confirm that, If {...}, skip / do not / do, or only (if / when / after)`.

Cross-references (used in Pass 3): `(see / refer to / as in / as per / repeat) step N, anywhere in a paragraph`, captures the referenced step number *N*.

Imperative-verb fallback (kind `STEP`, no extractable step number): for paragraphs no longer than 240 characters that did not match any of the patterns above, the first word is checked against a fixed lexicon: *open, close, turn, press, hold, release, wait, check, verify, confirm, record, measure, set, remove, replace, install, connect, disconnect, wear, don, doff, isolate, lockout, tagout, drain, fill, rinse, clean, wipe, apply, begin, stop, start, shut, energize, deenergize*. The 240-character cap and the verb whitelist together suppress the most common false positive (long body paragraphs that incidentally start with an imperative).

The four pattern families are matched in the order listed (`HEADING > STEP > SAFETY > PRECONDITION`), and the first match wins. Any paragraph that matches none of the patterns is labelled kind `BODY` and attached to the most recent `STEP` during Pass 2 packing.

S2. Token budget and pack constants

The reference implementation targets 512 tokens per chunk (`TARGET_TOKENS`) and enforces a hard ceiling of 900 tokens (`HARD_MAX_TOKENS`) above which a step group is split at the lowest-cost boundary respecting P1–P3. Token counts are approximated by a 4-characters-per-token heuristic on the raw text. Both constants are exposed at module level in the released code.

References

1. Brown, T. B. *et al.* Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 1877–1901 (2020).
2. Friedrich, F., Mendling, J. & Puhlmann, F. Process model generation from natural language text. In *Advanced Information Systems Engineering (CAiSE 2011)*, vol. 6741 of *Lecture Notes in Computer Science*, 482–496, DOI: [10.1007/978-3-642-21640-4_36](https://doi.org/10.1007/978-3-642-21640-4_36) (Springer, 2011).
3. van der Aa, H., Carmona, J., Leopold, H., Mendling, J. & Padró, L. Challenges and opportunities of applying natural language processing in business process management. In *Proceedings of the 27th International Conference on Computational Linguistics (COLING)*, 2791–2801 (2018).

4. Bellan, P., Dragoni, M. & Ghidini, C. Extracting business process entities and relations from text using pre-trained language models and in-context learning. In *Enterprise Design, Operations, and Computing (EDOC 2022)*, vol. 13585 of *Lecture Notes in Computer Science*, 182–199, DOI: [10.1007/978-3-031-17604-3_11](https://doi.org/10.1007/978-3-031-17604-3_11) (Springer, 2022).
5. Koupaei, M. & Wang, W. Y. WikiHow: A large scale text summarization dataset. *arXiv preprint arXiv:1810.09305* (2018).
6. Lewis, P. *et al.* Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 9459–9474 (2020).
7. Karpukhin, V. *et al.* Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 6769–6781, DOI: [10.18653/v1/2020.emnlp-main.550](https://doi.org/10.18653/v1/2020.emnlp-main.550) (2020).
8. Ji, Z. *et al.* Survey of hallucination in natural language generation. *ACM Comput. Surv.* **55**, 248:1–248:38, DOI: [10.1145/3571730](https://doi.org/10.1145/3571730) (2023).
9. Liu, N. F. *et al.* Lost in the middle: How language models use long contexts. *Transactions Assoc. for Comput. Linguist.* **12**, 157–173, DOI: [10.1162/tacl_a_00638](https://doi.org/10.1162/tacl_a_00638) (2024).
10. Gemini Team, Google. Gemini: A family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805* (2023).
11. Reimers, N. & Gurevych, I. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 3982–3992, DOI: [10.18653/v1/D19-1410](https://doi.org/10.18653/v1/D19-1410) (2019).
12. Maynez, J., Narayan, S., Bohnet, B. & McDonald, R. On faithfulness and factuality in abstractive summarization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 1906–1919, DOI: [10.18653/v1/2020.acl-main.173](https://doi.org/10.18653/v1/2020.acl-main.173) (2020).
13. Xu, Y. *et al.* LayoutLM: Pre-training of text and layout for document image understanding. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 1192–1200, DOI: [10.1145/3394486.3403172](https://doi.org/10.1145/3394486.3403172) (2020).
14. Blecher, L., Cucurull, G., Scialom, T. & Stojnic, R. Nougat: Neural optical understanding for academic documents. *arXiv preprint arXiv:2308.13418* (2023).
15. Wei, J. *et al.* Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 24824–24837 (2022).

Author contributions statement

G.S. conceived the study, formalised the structural-fidelity properties, implemented the position-aware chunker and the evaluation pipeline, assembled the corpus, ran the experiments, analysed the results, and wrote the manuscript.

Additional information

Competing interests: The author declares no competing interests.

Funding: This research received no external funding; it was conducted as independent research.